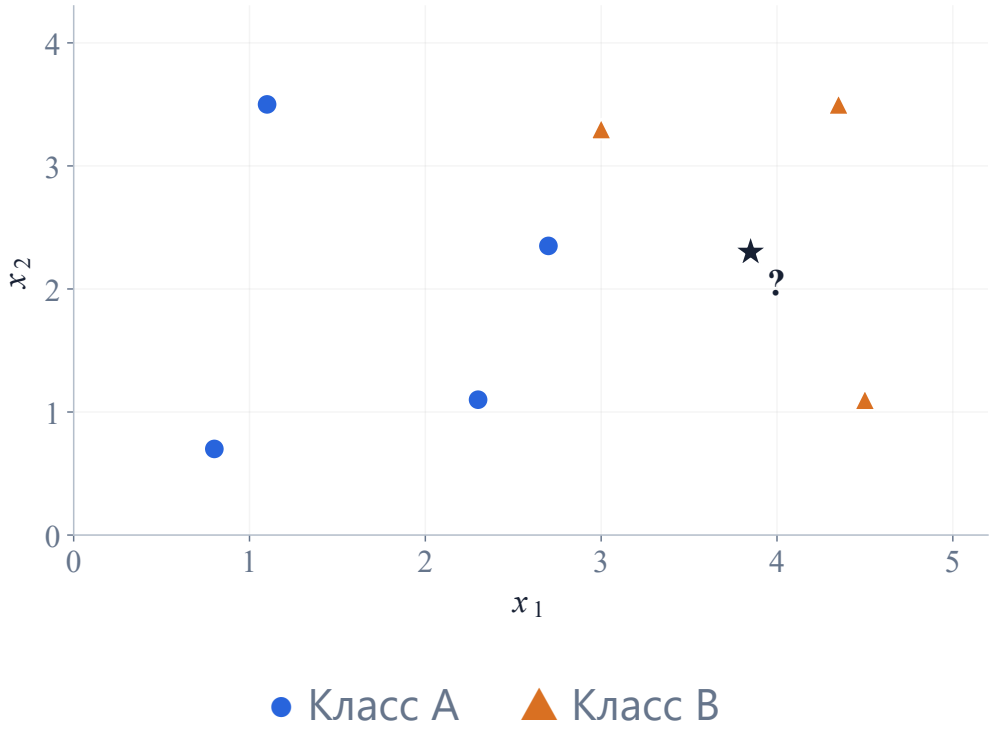
КЛАССИЧЕСКОЕ МАШИННОЕ ОБУЧЕНИЕ · НЕДЕЛЯ 7

Метод ближайших соседей и многоклассовая классификация

Как похожесть объектов превращается в прогноз



Что у нас уже есть

Линейная модель

Коэффициенты задают общую зависимость.

Логистическая регрессия

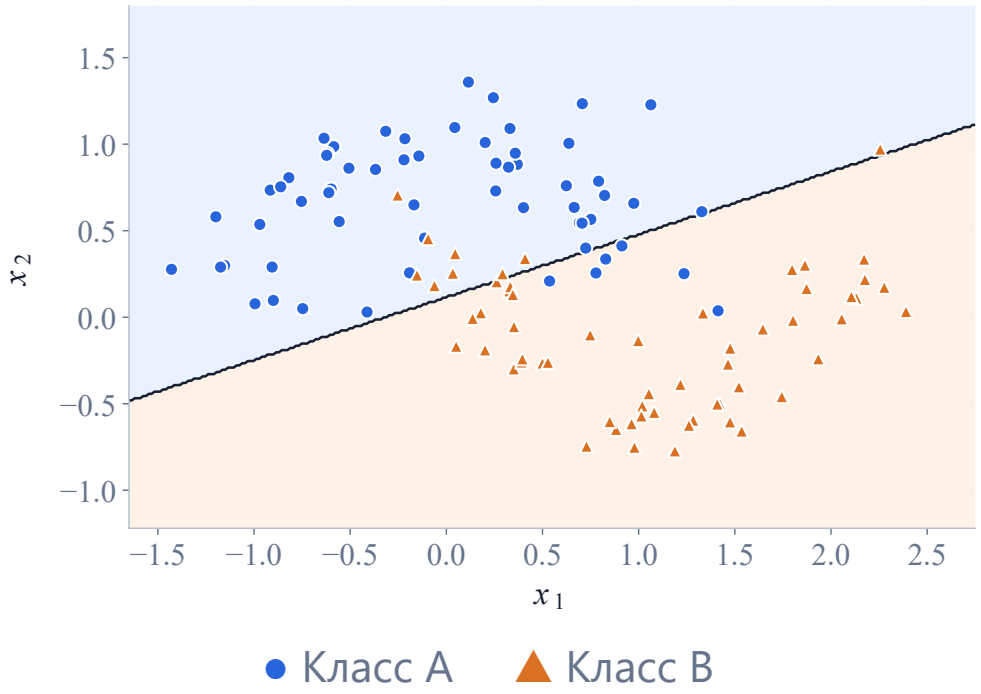
Score превращается в вероятность.

Кросс-валидация

Проверяем качество настройки.

Можно ли делать прогноз, не подбирая одну глобальную формулу?

Линейная граница и сложная геометрия классов



В исходных признаках одной прямой недостаточно.

Преобразования признаков могут изменить ситуацию.

А если известны метки ближайших наблюдений?

Предсказать по одному ближайшему объекту

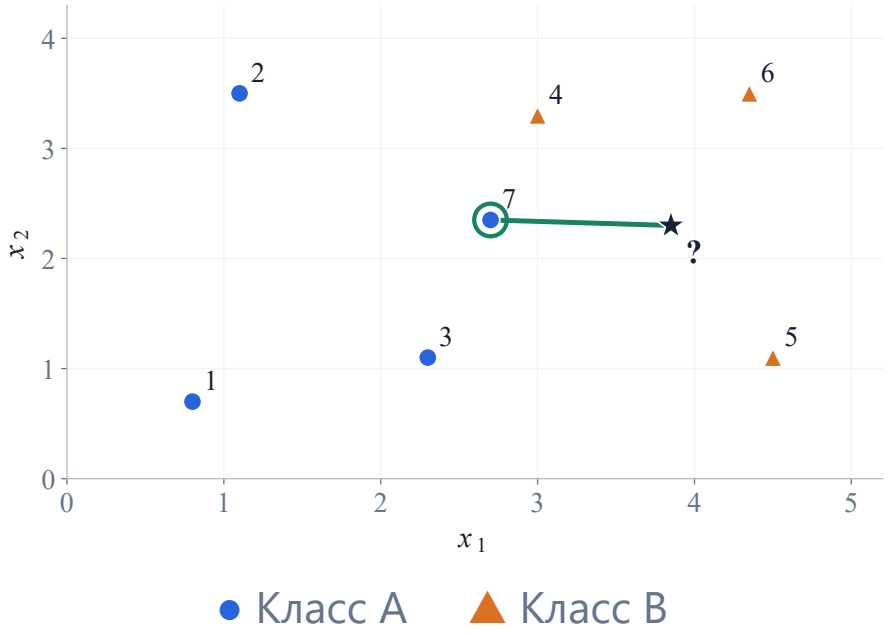


Обучающие точки имеют известные классы.

Какой класс назначить новой точке?

Ближайший — объект №7, класс A.
Прогноз новой точки — A.

Запишем 1NN математически



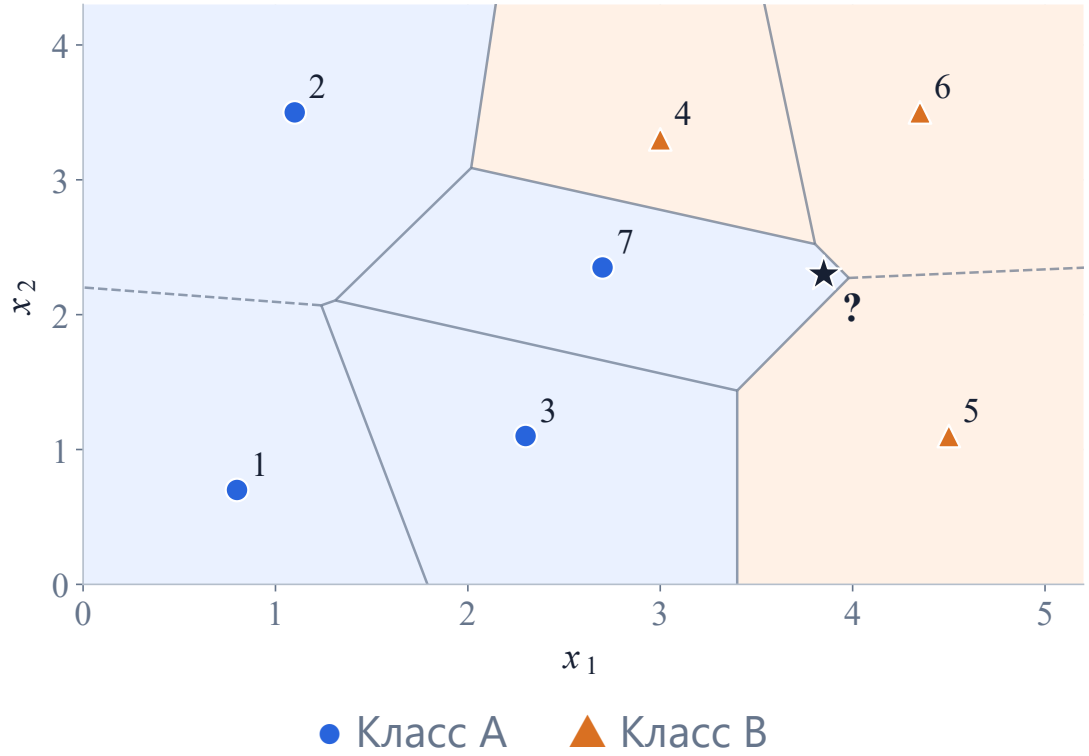
$$i^*(x) \in \arg\min_{i=1,\dots,n} d(x, x_i)$$

Ищем **индекс** ближайшего объекта.

$$\hat{y}(x) = y_{i^*(x)}$$

x — новый объект; i — номер обучающего; d — расстояние. Для другого объекта ближайший индекс может быть другим.

От ближайшей точки к областям Вороного



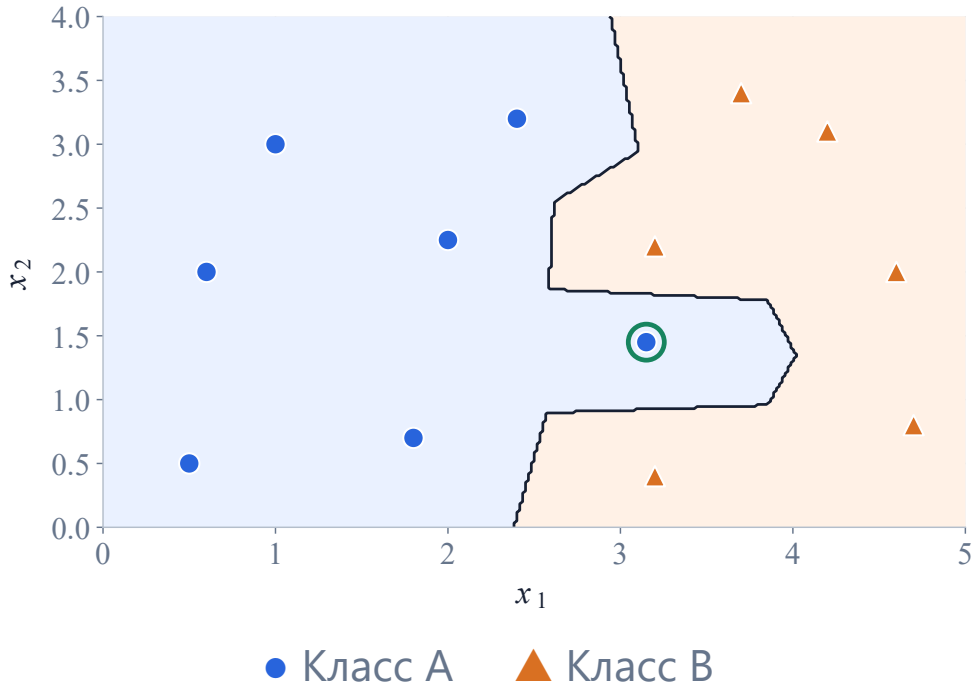
Где ближайший объект остаётся тем же?

Все такие точки образуют его ячейку Вороного.

Каждому наблюдению — своя ячейка.

Граница классов проходит между ячейками разных классов.

Цена избыточной локальности

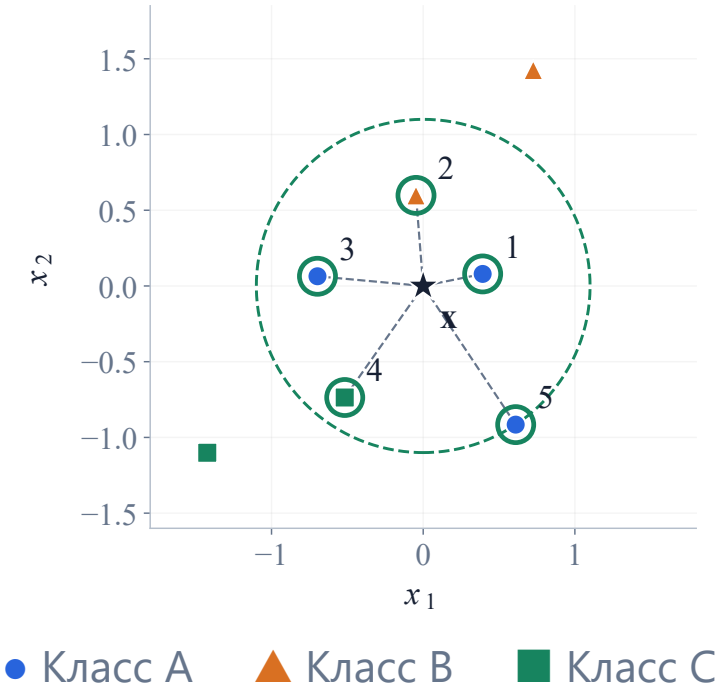


Одна обучающая точка получает собственную область решений.

Случайная ошибка разметки может изменить прогнозы вокруг неё.

Идеальная подстройка к train не гарантирует устойчивых новых прогнозов.

Не один, а k ближайших



$$k = 5$$

Каждый выбранный сосед голосует за свой класс.

Класс с наибольшим числом голосов становится прогнозом.

Правило голосования

$$\mathcal{N}_k(x) \text{ — индексы } k \text{ ближайших обучающих объектов}$$

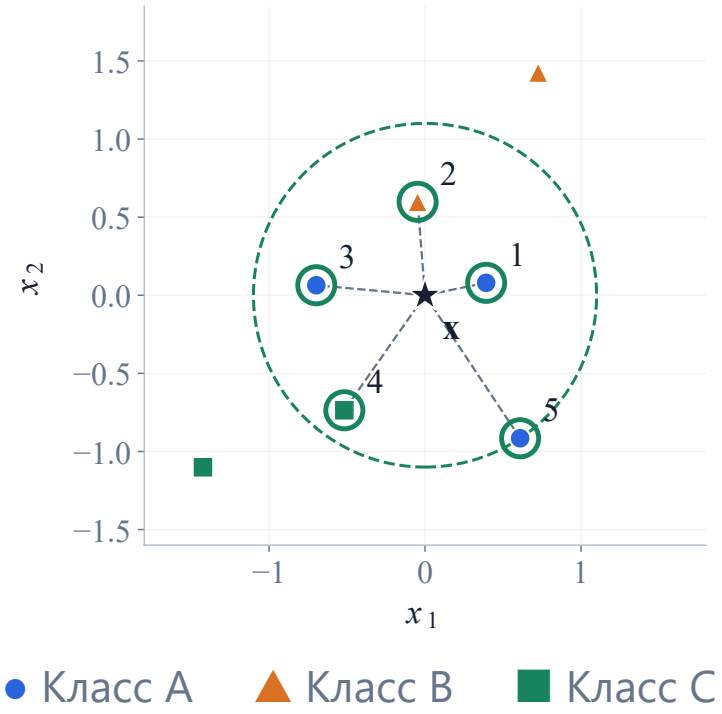
$$V_c(x) = \sum_{i \in \mathcal{N}_k(x)} \mathbb{I}(y_i = c)$$

Индикатор равен 1 для соседа класса c и 0 для остальных. Сумма — голоса за класс.

$$\hat{y}(x) \in \arg\max_{c \in \mathcal{Y}} V_c(x), \qquad \mathcal{Y} = \{1, \ldots, K\}$$

Argmax выбирает класс-победитель. Для ничьей нужно заранее заданное правило.

Одно голосование, три класса



Сосед	Расстояние	Класс
1	0.4	A
2	0.6	B
3	0.7	A
4	0.9	C
5	1.1	A

$$(V_A, V_B, V_C) = (3, 1, 1) \Rightarrow \hat{y} = A$$

Локальные доли классов

$$\hat{p}_c(x) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x)} \mathbb{I}(y_i = c)$$

$$(3, 1, 1) \xrightarrow{\div 5} (\hat{p}_A, \hat{p}_B, \hat{p}_C) = (0.6, 0.2, 0.2)$$

0.6 означает: 60% выбранных соседей относятся к A.

Доли классов — оценки условных вероятностей. Их качество проверяется на новых данных.

Что происходит при fit и predict

Логистическая регрессия

`fit`: оценить коэффициенты.

`predict`: применить общую функцию.

kNN

`fit`: сохранить train, подготовить поиск.

`predict`: расстояния → соседи → голоса.

Обучение по примерам: существенная часть работы приходится на прогноз.

Instance-based / lazy learning. Настройки kNN: число соседей, расстояние и веса.

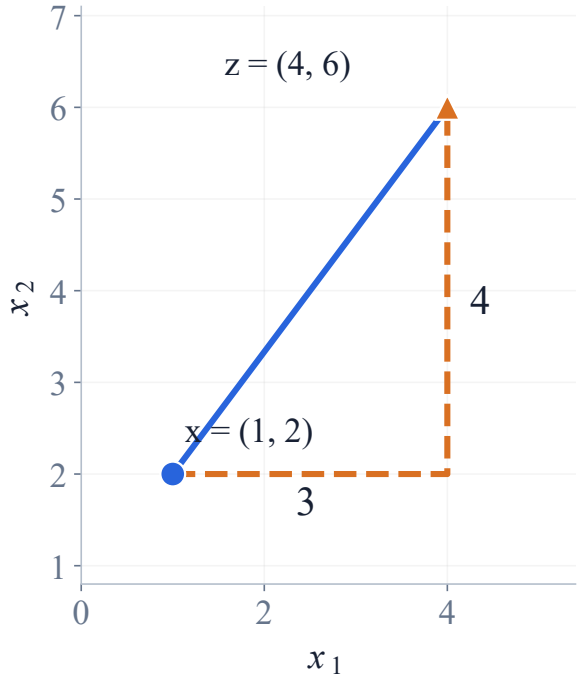
Какие объекты действительно близки?

**Похожи по длине текста — но различаются по времени обращения.
Или наоборот. Какую пару считать более похожей?**

Формула расстояния задаёт, какие различия признаков существенны для поиска соседей.

Разные способы измерять близость могут выбрать разных соседей для одного объекта.

Евклидово расстояние

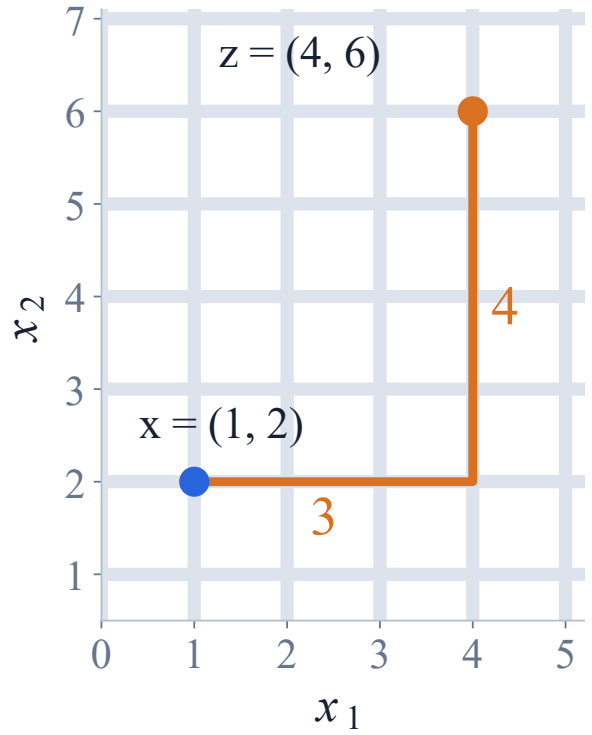


$$d_2(x, z) = \sqrt{\sum_{j=1}^{p}(x_j - z_j)^2}$$

$$d_2 = \sqrt{3^2 + 4^2} = 5$$

Привычная длина отрезка между точками.

Манхэттенское расстояние

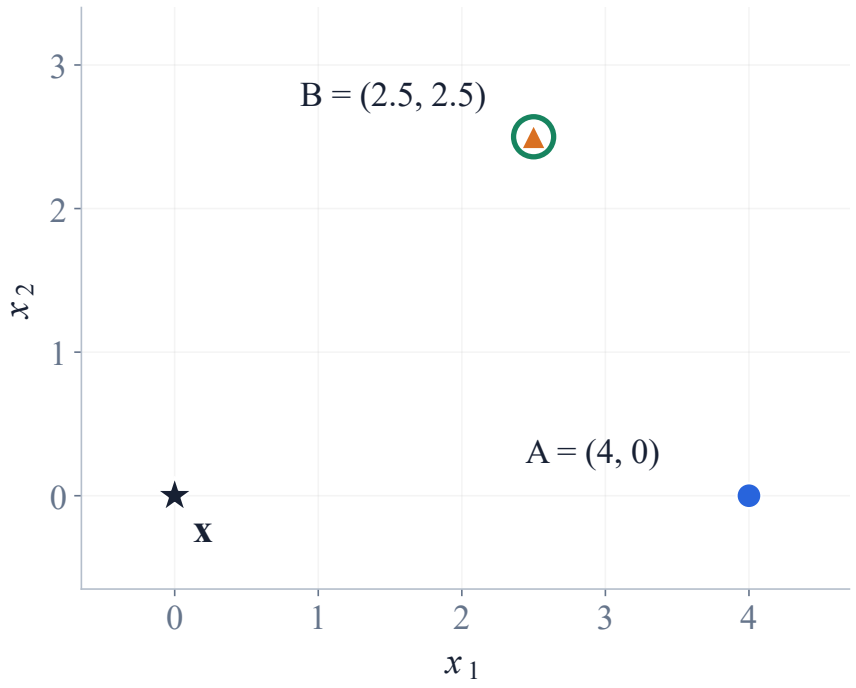


По прямоугольной сетке улиц идём вдоль кварталов, без движения наискосок.

$$d_1(x, z) = \sum_{j=1}^{p} |x_j - z_j|$$

$$d_1 = 3 + 4 = 7$$

Разные расстояния — разные соседи



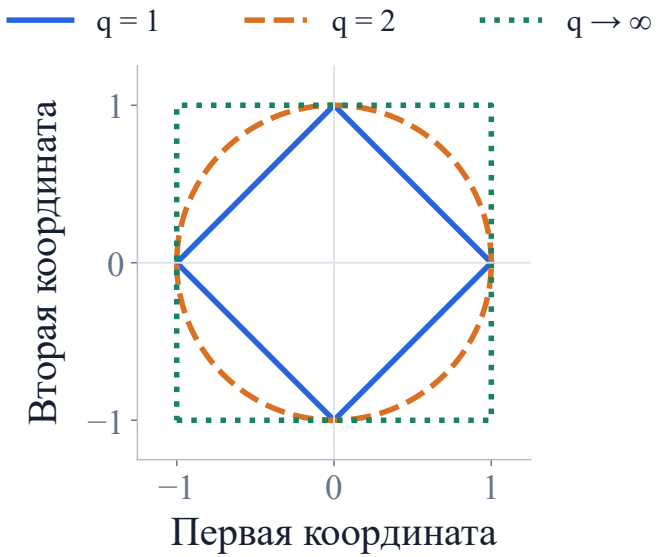
$$x = (0, 0),$$
$$A = (4, 0),$$
$$B = (2.5, 2.5).$$

	До A	До B
L1	4	5
L2	4	≈ 3.54

Выбор метрики — часть задания модели.

Семейство расстояний Минковского

$$d_q(x, z) = \left(\sum_{j=1}^{p} |x_j - z_j|^q \right)^{1/q}, \qquad q \geq 1$$



q = 1: манхэттенское.

q = 2: евклидово.

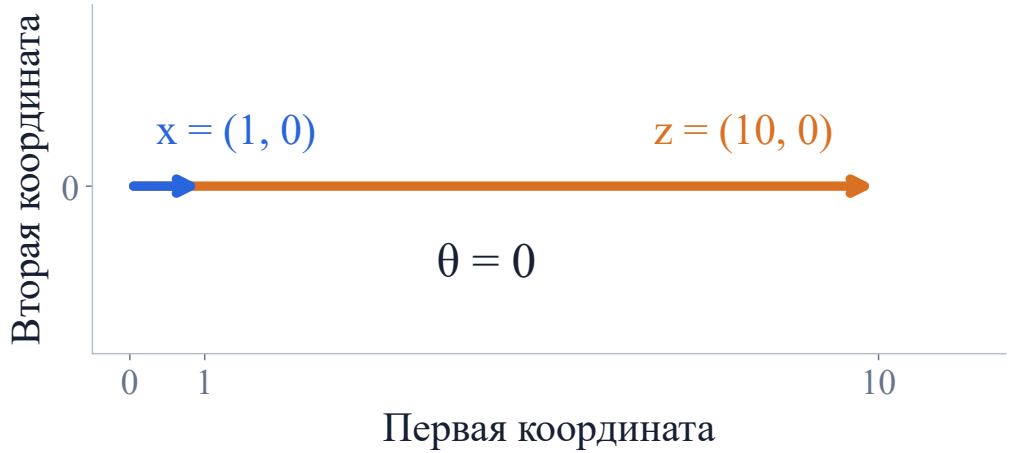
q → ∞: расстояние Чебышёва.

$$d_\infty(x, z) = \max_j |x_j - z_j|$$

Определяется наибольшим различием координат.

Когда важнее направление, чем длина

Тексты разного объёма могут иметь похожие относительные частоты слов. Сравним направления их векторов.



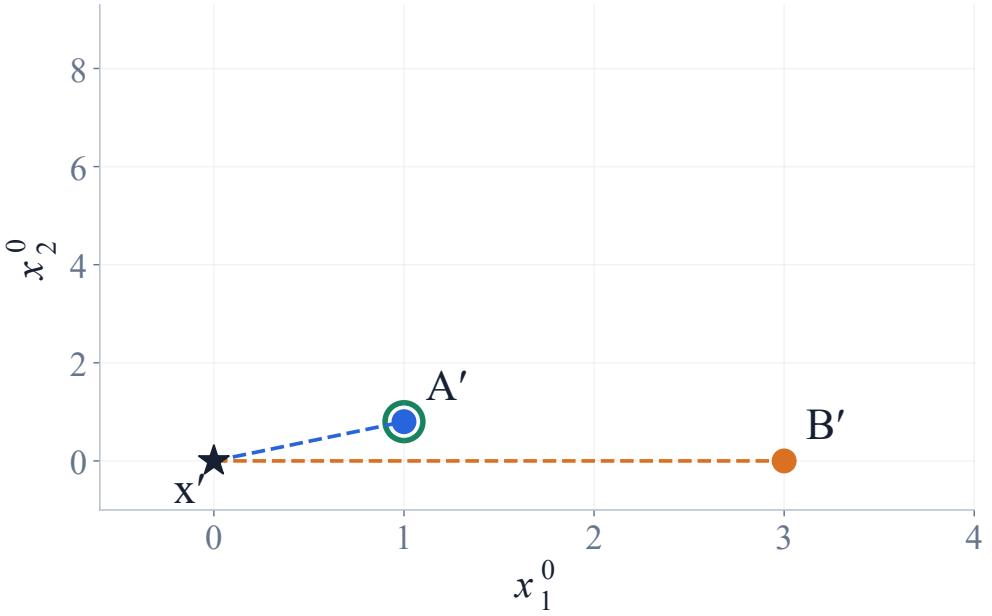
$$\mathrm{sim}_{cos}(x, z) = \frac{x^\top z}{\|x\|_2 \|z\|_2} = \cos\theta$$

Для этих двух векторов сходство равно 1.

$$\mathrm{dist}_{cos} = 1 - \mathrm{sim}_{cos}$$

Расстояние: меньше — ближе. Сходство: больше — похожее направление.

Масштаб меняет ближайшего соседа



$$x' = (0, 0),\ A' = (1, 0.8),\ B' = (3, 0)$$

$$d(x', A') = \sqrt{1.64} \approx 1.28$$

$$d(x', B') = 3$$

Ближайший: A′.

Объекты те же. Смена единиц измерения может изменить прогноз kNN.

StandardScaler: различия относительно разброса

$$z = \frac{x - \mu_{\text{train}}}{s_{\text{train}}}$$



В kNN разность по каждой координате теперь измеряется в единицах её обучающего стандартного отклонения.

Статистики — только по train, внутри CV — по train-фолду. При выбросах используют и другие способы, например RobustScaler.

MinMaxScaler: обучающий диапазон в [0,1]

$$x_{\text{scaled}} = \frac{x - \min_{\text{train}}}{\max_{\text{train}} - \min_{\text{train}}}$$



$$x_{\text{new}} = 40 \quad \Rightarrow \quad \frac{40 - 10}{30 - 10} = 1.5$$

Новые значения могут выходить за [0,1]. Минимум и максимум берём только по соответствующей обучающей части.

От одинаковых голосов — к весам соседей

Более близкому соседу можно дать больший вес. Для выбранных k соседей задаём неотрицательные веса:

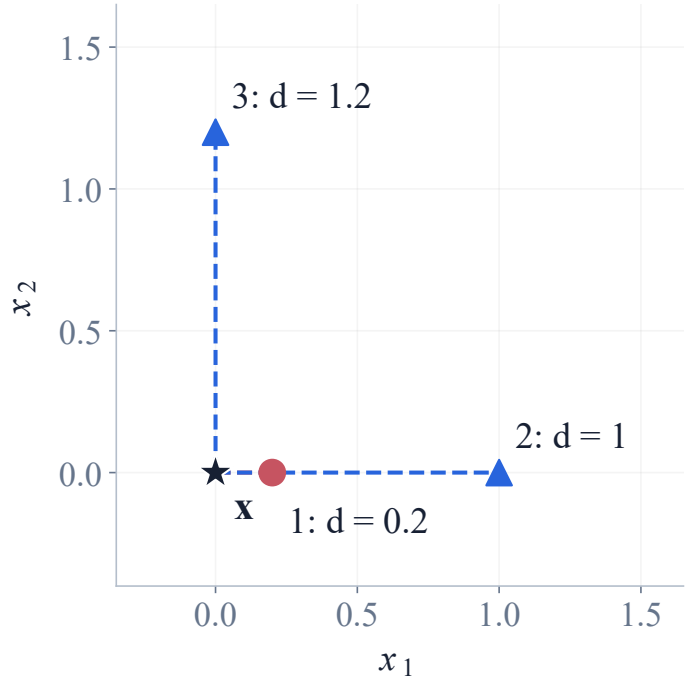
$$w_i(x) \geq 0, \qquad \sum_{i \in \mathcal{N}_k(x)} w_i(x) > 0$$

$$\hat{p}_c(x) = \frac{\sum_{i \in \mathcal{N}_k(x)} w_i(x) \mathbb{I}(y_i = c)}{\sum_{i \in \mathcal{N}_k(x)} w_i(x)}$$

Сверху — вес соседей класса c. Снизу — вес всех выбранных соседей.

Одинаковые веса возвращают обычную долю голосов.

Обычные и взвешенные голоса



Без весов: красный — 1 голос, синий — 2.

$$w_i(x) = \frac{1}{d(x, x_i)}$$

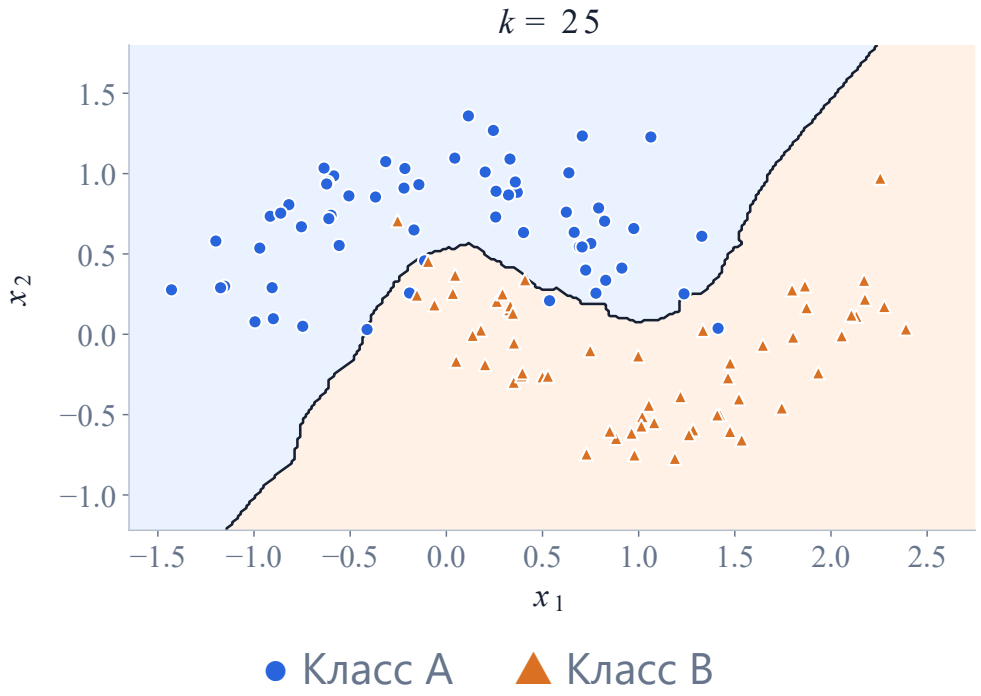
$$V_{red} = 5,$$

$$V_{blue} = 1 + \frac{1}{1.2} \approx 1.83.$$

Близкий красный сосед перевесил двух дальних синих.

При ненулевых расстояниях. Нулевые расстояния требуют отдельного правила.

Что меняет число k?



Малое k: сильнее реакция на отдельные точки.

Больше k: сглаживаем локальное голосование.

Слишком большое k может скрыть полезную структуру.

Управляем локальностью и гибкостью. Качество не обязано меняться монотонно.

Выбрать k по валидации

$$k \in \{1, 3, 5, 7, 11, 15, 21\}$$

Те же CV-разбиения → Обучение и прогнозы → Сравнение метрики

Почему train accuracy подталкивает к k = 1?

Если сама train-точка включена в поиск, она обычно оказывается своим ближайшим соседом. Проверка на новых объектах снимает это преимущество.

Подготовка признаков обучается внутри train-фолда. Финальный test остаётся отдельно.

Близкие соседи должны ещё и найтись

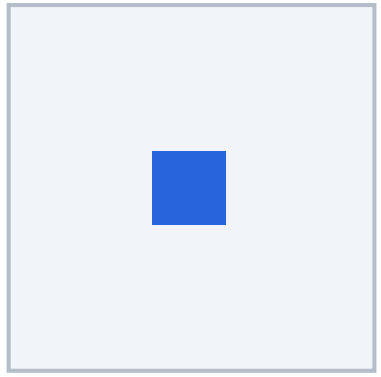
Хотим узкую окрестность и достаточно наблюдений в ней. При той же относительной стороне её доля пространства быстро уменьшается.

1 признак



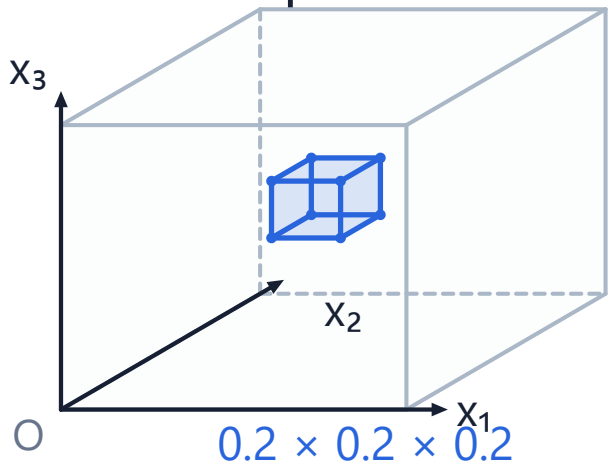
$r = 0.2 \to 20\%$

2 признака



$r^2 = 0.04 \to 4\%$

3 признака

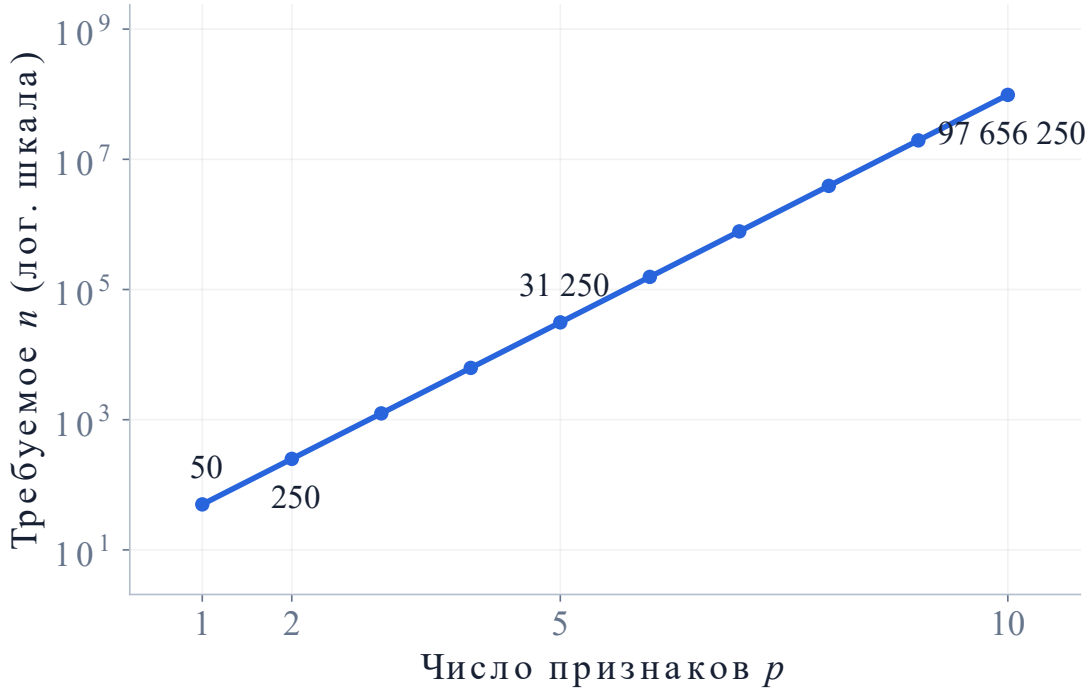


$0.2 \times 0.2 \times 0.2$

$r^3 = 0.008 \to 0.8\%$

Идеализируем: равномерное распределение в единичном пространстве; вся окрестность лежит внутри него.

Сколько данных нужно для десяти соседей?



$$\mathbb{E}[N_{\text{inside}}] = nr^p$$

$$r = 0.2, \quad n \approx \frac{10}{0.2^p}$$

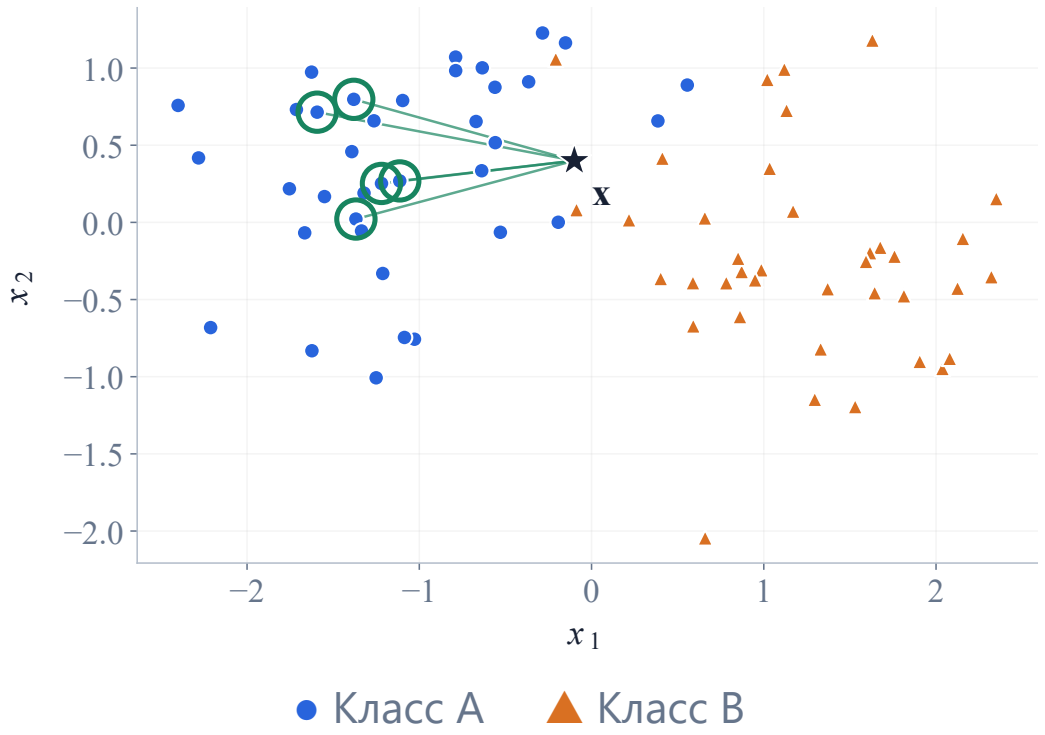
p = 2: нужно 250.

p = 10: 97 656 250.

Если данных мало, kNN найдёт k соседей за счёт более широкой окрестности — ценой локальности.

Числа относятся к равномерному примеру, а не к любым данным с p признаками.

Шумовые координаты меняют само соседство



2 полезные + 30 шумовых

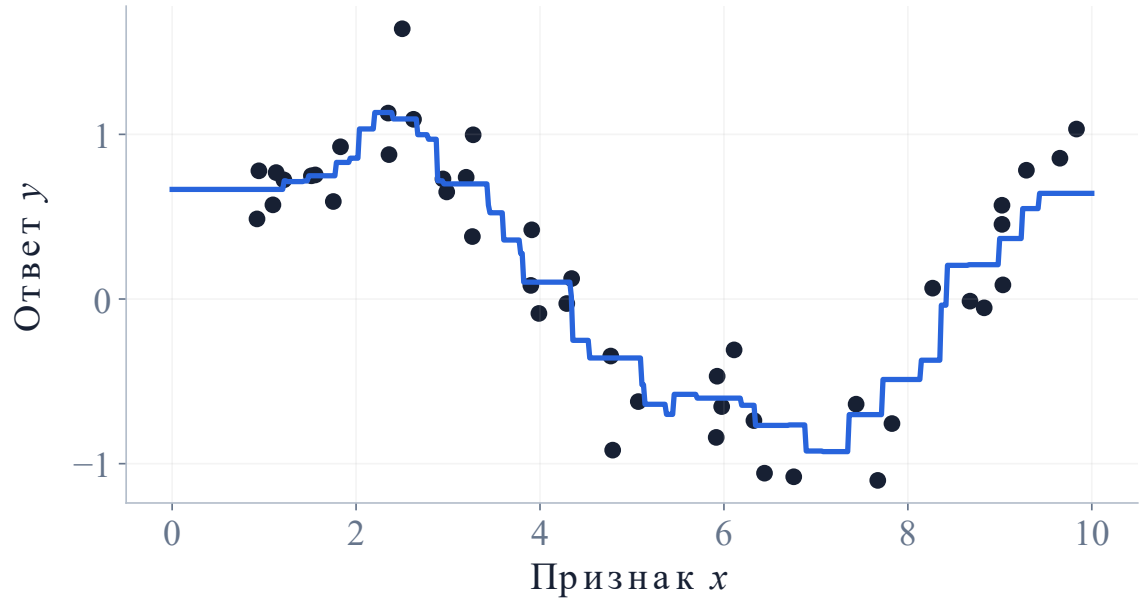
Кольца и отрезки отмечают выбранных соседей.

На рисунке всегда показаны только первые две координаты.

Стандартизация меняет масштаб, но не устраняет нерелевантность признаков.

KD-tree и BallTree — структуры для ускорения поиска соседей. Их выигрыш зависит от геометрии и размерности данных.

Тот же механизм работает в регрессии



$$\hat{f}(x) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x)} y_i$$

Классы голосуют.

Числовые ответы усредняются.

$$(40 + 45 + 50)/3 = 45$$

Цена локальности: fit сохраняет примеры; predict ищет соседей среди train.

От kNN к многоклассовой постановке

kNN уже выбирает между A, B и C.
Как расширить бинарную логистическую регрессию?

$$Y \in \{1, \ldots, K\}$$

В многоклассовой классификации объекту соответствует **одна метка** из K взаимоисключающих вариантов.

Что должна выдавать многоклассовая модель

A · Оплата

Списание денег

B · Доставка

Движение заказа

C · Техподдержка

Ошибка сервиса

Для выбора класса достаточно оценок. В вероятностной модели хотим получить согласованный вектор классовых вероятностей:

$$x \longrightarrow (p_A(x), p_B(x), p_C(x)) \longrightarrow \hat{y}(x)$$

$$p_c(x) \geq 0, \qquad \sum_{c=1}^{K} p_c(x) = 1, \qquad \hat{y}(x) \in \arg\max_c p_c(x)$$

Начнём со знакомого бинарного классификатора

Можно ли построить многоклассовую модель из уже знакомых бинарных классификаторов?

Для обращения нужен один из трёх отделов. Но мы уже умеем отвечать на вопрос «относится ли оно к A?».

Поставим такой вопрос отдельно для каждого класса.

One-vs-Rest: A против остальных

Исходный класс	Смысл бинарного ответа	Новая метка
A · Оплата	A	1
B · Доставка	Не-A	0
C · Техническая проблема	Не-A	0

На тех же признаках обучаем знакомую бинарную модель: A против всех остальных.

Три самостоятельные бинарные модели

Истинный класс	A vs rest	B vs rest	C vs rest
A	1	0	0
B	0	1	0
C	0	0	1

Признаки объектов одни и те же; для каждой задачи меняем бинарные ответы.

Для K классов обучаем K отдельных классификаторов.

Из трёх scores в одну метку

Объект	Score A	Score B	Score C	Прогноз
1	1.4	0.3	−0.8	A
2	−0.2	1.8	0.6	B
3	0.2	−0.1	1.1	C

$$\hat{y}(x) \in \arg\max_{c} s_c(x)$$

Для бинарных логистических моделей сравнение сигмоид даёт тот же порядок: сигмоида строго возрастает.

Три бинарные вероятности ещё не распределение

$$(\hat{p}_A^{bin}, \hat{p}_B^{bin}, \hat{p}_C^{bin}) = (0.8, 0.7, 0.3), \qquad \sum_c \hat{p}_c^{bin} = 1.8$$

У объекта один правильный класс. Можно ли считать эти числа его распределением?

$$\hat{p}_c^{OvR} = \frac{\hat{p}_c^{bin}}{\sum_j \hat{p}_j^{bin}} \quad \Rightarrow \quad (0.444, 0.389, 0.167)$$

Нормировка объединяет выходы. Параметры бинарных моделей всё ещё обучались независимо.

Что мы получили с помощью OvR

- Обучили K независимых бинарных классификаторов.
- По их оценкам выбираем один итоговый класс.
- Бинарные вероятности могут не давать сумму 1; нормировка не меняет способа обучения.

Можно ли обучить одну модель, в которой все классы учитываются совместно?

По одной линейной оценке для каждого класса

Один объект описан двумя признаками. Каждый класс получает свою строку коэффициентов.

$$s_A(x) = w_{A1}x_1 + w_{A2}x_2 + b_A,$$

$$s_B(x) = w_{B1}x_1 + w_{B2}x_2 + b_B,$$

$$s_C(x) = w_{C1}x_1 + w_{C2}x_2 + b_C.$$

Три оценки одного объекта — параметры одной многоклассовой модели. Найдём их по общей функции потерь.

Соберём коэффициенты в матрицу W

$$W = \begin{pmatrix} w_{A1} & w_{A2} \\ w_{B1} & w_{B2} \\ w_{C1} & w_{C2} \end{pmatrix}, \qquad b = \begin{pmatrix} b_A \\ b_B \\ b_C \end{pmatrix}$$

$$\underbrace{s(x)}_{K\times 1} = \underbrace{W}_{K\times p}\ \underbrace{x}_{p\times 1} + \underbrace{b}_{K\times 1}$$

Строка W — коэффициенты одного класса. Столбец W — влияние одного признака на все классовые оценки.

Посчитаем три оценки одного объекта

$$W = \begin{pmatrix} 1 & 0.5 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad b = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, \quad x = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

$$s_A = 1 \cdot 1 + 0.5 \cdot 2 = 2,$$
$$s_B = 1 \cdot 1 + 0 \cdot 2 = 1,$$
$$s_C = 0.$$

$$s(x) = Wx + b = \begin{pmatrix} 2 \\ 1 \\ 0 \end{pmatrix}$$

Один набор коэффициентов применяется ко всем объектам. Меняется x — меняются scores.

Почему нельзя просто нормировать scores?

Попробуем разделить каждую линейную оценку на сумму оценок:

$$(s_A, s_B, s_C) = (2, 1, -1), \qquad \sum_c s_c = 2$$

$$\frac{(2, 1, -1)}{2} = (1, 0.5, -0.5)$$

Сумма равна 1, но вероятность C отрицательна. Нужен переход от любых scores к положительным значениям.

Экспоненты дают положительные веса классов

$$p_c = \frac{e^{s_c}}{\sum_j e^{s_j}} > 0$$

$$s_A > s_B \Rightarrow e^{s_A} > e^{s_B}$$

Нормируем положительные веса: сумма р равна 1.

Порядок оценок сохраняется.

$$\frac{p_A(x)}{p_B(x)} = \frac{e^{s_A(x)}}{e^{s_B(x)}} = e^{s_A(x) - s_B(x)}$$

$$\boxed{\log \frac{p_A(x)}{p_B(x)} = s_A(x) - s_B(x)}$$

Логарифм отношения вероятностей снова линейно связан с признаками.

Softmax: одна формула для всех классов

$$p_c(x) = \frac{e^{s_c(x)}}{\sum_{j=1}^{K} e^{s_j(x)}}$$

Свой числитель.

Общий знаменатель.

Класс	Score	Экспонента	Вероятность
A	2	7.389	0.665
B	1	2.718	0.245
C	0	1.000	0.090

Общий знаменатель: 11.107

Сумма вероятностей равна 1. Общий знаменатель связывает все классы.

Softmax продолжает сигмоиду

$$p_1 = \frac{e^{s_1}}{e^{s_0} + e^{s_1}} = \frac{1}{1 + e^{s_0 - s_1}}$$

$$\boxed{p_1 = \sigma(s_1 - s_0)}$$

$$s_1(x) - s_0(x) = (w_1 - w_0)^\top x + (b_1 - b_0)$$

Сигмоида применяется к разности двух scores. Эта разность сама линейна по признакам.

Вспомним бинарную log loss

$$L_{bin}(y, p) = -y \log p - \big(1 - y\big) \log(1 - p)$$

$$y = 1: \quad L = -\log p \qquad\qquad y = 0: \quad L = -\log(1 - p)$$

Берём отрицательный логарифм вероятности фактически наблюдённого ответа.

Cross-entropy одного многоклассового объекта

Модель выдала вероятности трёх классов. Истинный класс объекта — **B**.

A	**B · истинный класс**	C
≈ 0.665	**≈ 0.245**	≈ 0.090

$$L_i = -\log p_B \approx 1.408$$

$$\boxed{L_i = -\log p_{y_i}(x_i)}$$

Чем меньшую вероятность модель назначает правильному классу, тем больше значение функции потерь.

Почему cross-entropy подходит для вероятностей

Предположим, среди объектов с одними и теми же признаками 70% относятся к A, 20% — к B и 10% — к C.

Реальные доли классов

A · 70% B · 20% C · 10%

Идеальный вероятностный прогноз

$$(p_A, p_B, p_C) = (0.7, 0.2, 0.1)$$

Если свободно выбирать вероятности, такой прогноз даёт наименьшую среднюю log loss при многократном повторении этих условий.

Softmax обеспечивает положительность и сумму 1. Соответствие реальным долям зависит от обученной модели и данных.

Линейная форма scores ограничивает модель, а обучающая выборка конечна. Поэтому найденные вероятности могут отличаться от реальных долей.

Меняем score A — меняется loss объекта B

$$p_B = \frac{e^{s_B}}{e^{s_A} + e^{s_B} + e^{s_C}}, \qquad y_i = B$$

До: scores (2, 1, 0)

$$p_B \approx 0.245, \qquad L_B \approx 1.408$$

После: scores (3, 1, 0)

$$p_B \approx 0.114, \qquad L_B \approx 2.170$$

Увеличиваем только score A. Его экспонента увеличивает общий знаменатель.

Вероятность каждого класса зависит от коэффициентов всех классов. Поэтому параметры multinomial-модели обучаются совместно.

Одна целевая функция для всех коэффициентов

$$R_n(W, b) = -\frac{1}{n} \sum_{i=1}^{n} \log p_{y_i}(x_i; W, b)$$

Усредняем cross-entropy всех обучающих объектов.

$$(\hat{W}, \hat{b}) \in \arg\min_{W,b} R_n(W, b)$$

В одной минимизации участвуют все строки W и все свободные члены.

От признаков — к среднему риску

Признаки x	Истинный y	Scores	Прогноз	p_y	Loss
(1, 2)	B	(2, 1, 0)	A	0.245	1.408
(1, −2)	B	(0, 1, 0)	B	0.576	0.551
(−2, 0)	C	(−2, −2, 0)	C	0.787	0.240
(2, 2)	A	(3, 2, 0)	A	0.705	0.349

$$R_4(W, b) \approx \frac{1.408 + 0.551 + 0.240 + 0.349}{4} = 0.637$$

Меняем W,b — пересчитываются scores, вероятности и потери всей выборки.

Что значит обучить многоклассовую модель

W, b → Линейные scores → Softmax → Средняя cross-entropy

Коэффициенты всех классов — параметры одной модели. Softmax связывает вероятности, поэтому общая функция потерь зависит от всего набора W,b.

Численный оптимизатор подбирает все параметры совместно. Можно добавить знакомый штраф на коэффициенты.

OvR: K отдельных бинарных задач.

Multinomial: одна совместная задача.

Например, для совместной минимизации используют L-BFGS.

Матрица ошибок в общей форме

Прогноз модели →

Истинный класс	A	B	C
A	C_{AA}	C_{AB}	C_{AC}
B	C_{BA}	C_{BB}	C_{BC}
C	C_{CA}	C_{CB}	C_{CC}

Строки — истинные классы.
Столбцы — прогноз модели.

$$TP_C = C_{CC}$$

$$FP_C = C_{AC} + C_{BC}$$

$$FN_C = C_{CA} + C_{CB}$$

Выбрали C положительным классом, остальные — отрицательными. Для любого другого класса делаем то же самое.

Одна матрица: 60 обращений в поддержку

Прогноз модели →

Истинный класс	A	B	C
A	38	2	0
B	4	10	1
C	2	2	1

Истинных объектов:
A — 40, B — 15, C — 5.

$$Accuracy = \frac{38 + 10 + 1}{60} = \frac{49}{60} \approx 0.817$$

Хорошо ли модель работает для каждого отдела?

Для класса C: кто TP, FP и FN?

Прогноз модели →

Истинный класс	A	B	C
A	38	2	0
B	4	10	1
C	2	2	1

Считаем C положительным классом.

$$TP_C = 1$$

$$FP_C = 0 + 1 = 1$$

$$FN_C = 2 + 2 = 4$$

Precision, recall и F1 для одного класса

Прогноз модели →

Истинный класс	A	B	C
A	38	2	0
B	4	10	1
C	2	2	1

$$Precision_C = \frac{1}{1+1} = 0.5$$

$$Recall_C = \frac{1}{1+4} = 0.2$$

$$F_{1,C} = \frac{2 \cdot 0.5 \cdot 0.2}{0.5 + 0.2} \approx 0.286$$

Precision: среди предсказанных C.

Recall: среди всех настоящих C.

Метрики по всем классам

Класс	Support	Precision	Recall	F1
A	40	0.864	0.950	0.905
B	15	0.714	0.667	0.690
C	5	0.500	0.200	0.286

Support — число объектов истинного класса: сумма его строки матрицы.

Общая accuracy ≈ 0.817 скрывает то, что найдена только пятая часть объектов C.

Macro: классы имеют одинаковый вес

$$F_1^{macro} = \frac{1}{K} \sum_{c=1}^{K} F_{1,c}$$

A · 1/3	B · 1/3	C · 1/3

$$F_1^{macro} \approx \frac{0.905 + 0.690 + 0.286}{3} \approx 0.627$$

C встречается в восемь раз реже A, но его вес в macro остаётся тем же.

Weighted: вес определяется долей класса

$$F_1^{weighted} = \sum_c \frac{n_c}{n} F_{1,c}$$

A · 40/60	B · 15/60	C

$$\frac{40 \cdot 0.905 + 15 \cdot 0.690 + 5 \cdot 0.286}{60} \approx 0.799$$

У крупных классов больше вес. Усредняем уже вычисленные классовые F1.

Одинаковые F1 — разные веса классов

Каждый класс весит 1/3

$$F_1^{macro} \approx 0.627$$

Вес пропорционален доле класса

$$F_1^{weighted} \approx 0.799$$

Значения F1 классов A, B, C остались прежними: 0.905, 0.690, 0.286. Изменились только веса при усреднении.

А если сначала объединить количества ошибок по всем классам и уже потом вычислить метрику?

Сначала объединяем TP, FP и FN

В macro и weighted сначала считали метрики классов. В micro сначала складываем количества по всем классам:

$$TP_{\text{all}} = \sum_{c=1}^{K} TP_c$$

$$FP_{\text{all}} = \sum_{c=1}^{K} FP_c$$

$$FN_{\text{all}} = \sum_{c=1}^{K} FN_c$$

Индекс all — итог по всем классам, а не имя отдельного класса.

Из этих общих количеств вычислим precision, recall и F1.

Соберём суммы из той же матрицы

Прогноз модели →

Истинный класс	A	B	C
A	38	2	0
B	4	10	1
C	2	2	1

60 объектов, прогнозы не меняются

$$TP_{\text{all}} = 38 + 10 + 1 = 49$$

$$FN_{\text{all}} = 2 + 5 + 4 = 11$$

$$FP_{\text{all}} = 6 + 4 + 1 = 11$$

По столбцам: ошибочно назначенные A, B и C.

Одна ошибка учитывается в FN и FP

Прогноз модели →

Истинный класс	A	B	C
A	38	2	0
B	4	10	1
C	2	2	1

Два обращения A ошибочно отправлены в B. Рассмотрим одно из них.

$$A \longrightarrow B$$

Для A: не нашли настоящий A — один FN.
Для B: ошибочно назначили B — один FP.

$$+1 \text{ в } FN_A, \qquad +1 \text{ в } FP_B$$

Это две записи об одном ошибочном прогнозе. Матрица остаётся прежней: 49 верных ответов и 11 ошибок.

От одной ошибки — к общему подсчёту

$$n_{\text{correct}} \text{ — верных}, \qquad n_{\text{errors}} = n - n_{\text{correct}} \text{ — ошибок}$$

Правильный прогноз

Один TP своему классу.

$$TP_{\text{all}} = n_{\text{correct}}$$

Неправильный прогноз

Один FN истинному классу и один FP предсказанному.

$$FP_{\text{all}} = FN_{\text{all}} = n_{\text{errors}}$$

У каждого объекта одна истинная и одна предсказанная метка. Считаем по всем классам.

Почему micro совпадает с accuracy

$$Precision^{micro} = \frac{TP_{\text{all}}}{TP_{\text{all}} + FP_{\text{all}}} = \frac{n_{\text{correct}}}{n}$$

$$Recall^{micro} = \frac{TP_{\text{all}}}{TP_{\text{all}} + FN_{\text{all}}} = \frac{n_{\text{correct}}}{n}$$

$$\boxed{Precision^{micro} = Recall^{micro} = F_1^{micro} = Accuracy}$$

В нашем примере: 49/60 ≈ 0.817. Micro отражает общий подсчёт верных и ошибочных ответов и не даёт отдельной информации о редких классах.

Для single-label multiclass с учётом всех классов. При равных precision и recall их F1 равно тому же числу.

Какую метрику выбрать для задачи?

Задача	Что важно узнать	Основной ориентир
Маршрутизация обращений	Какую долю направляем верно?	Accuracy при сопоставимой цене ошибок
Разные неисправности	Не теряем ли редкие типы?	Macro-F1 и результаты по классам
Критичный редкий исход	Сколько находим? Скольким сигналам верим?	Recall и precision этого класса

Как построить ROC-AUC, если классов больше двух?

Для бинарной ROC нужны истинные бинарные ответы и числовые scores. Softmax-модель уже выдаёт вероятности A, B и C.

Истинный класс	Вероятность A	A / не A
B	0.665	0
B	0.212	0
C	0.107	0
A	0.705	1

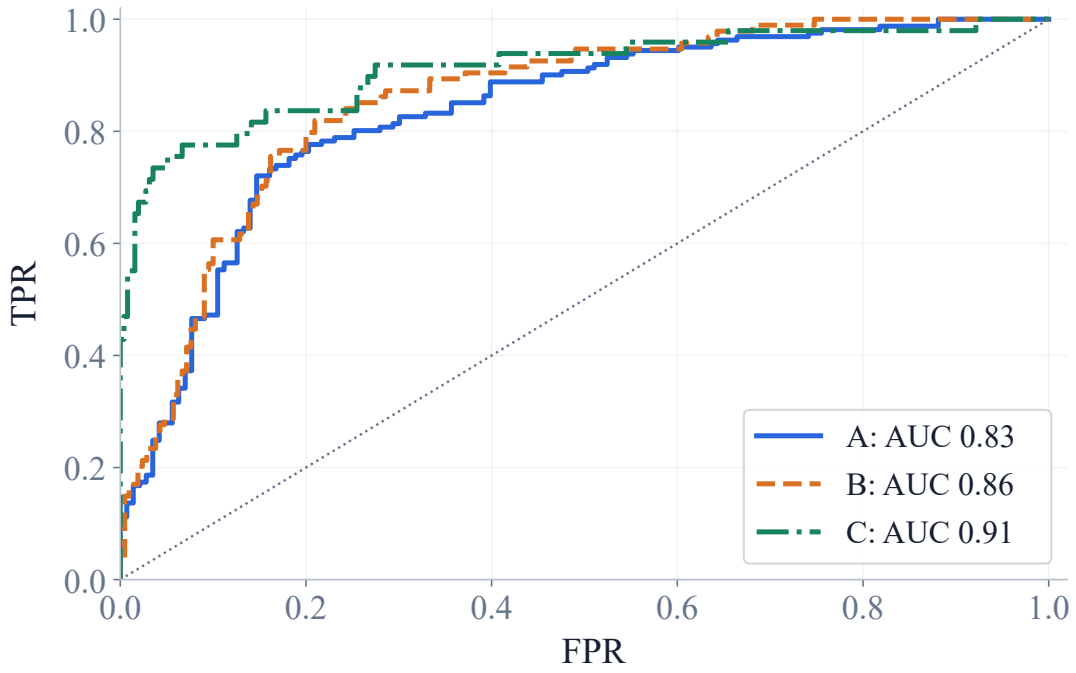
1. Отмечаем истинные ответы: A — 1, остальные — 0.

2. Берём вероятность A как score.
3. Строим бинарную ROC-кривую.

Оцениваем, насколько хорошо модель ранжирует объекты класса A.

Обучать отдельные OvR-классификаторы не нужно. Здесь OvR — способ оценивания выходов уже обученной многоклассовой модели.

Multiclass ROC-AUC через One-vs-Rest



Меняем порог по вероятности класса: получаем знакомые FPR и TPR.

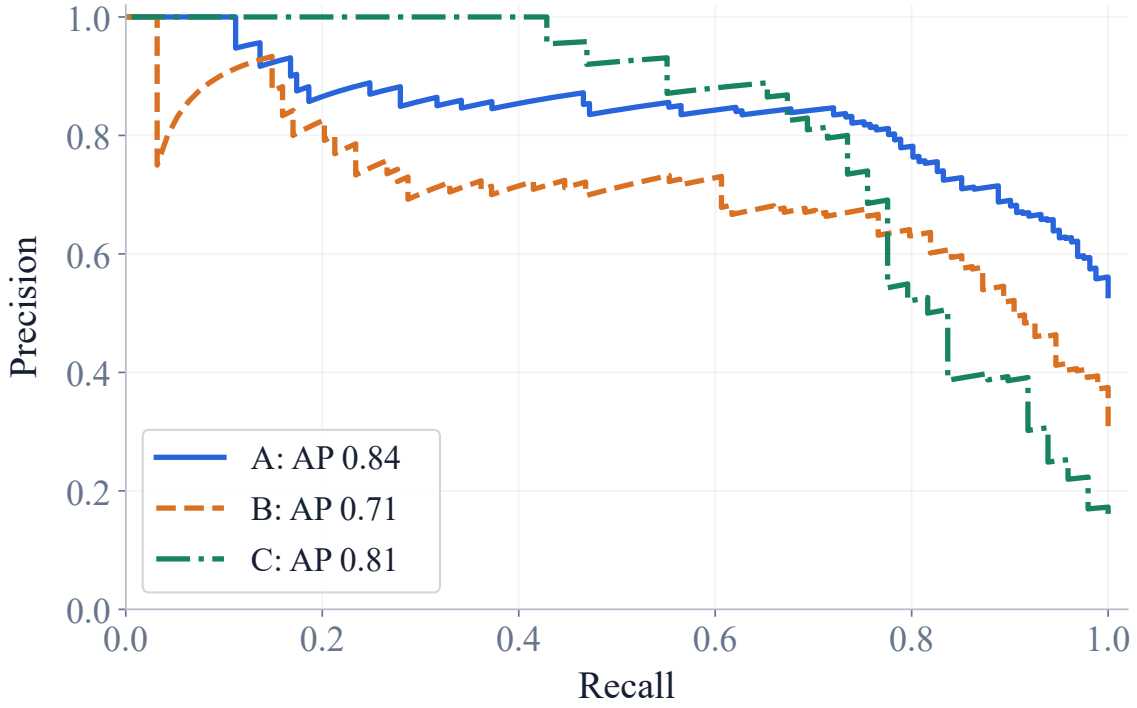
$$z_i^{(c)} = \mathbb{I}(y_i = c), \qquad p_c(x_i)$$

Отдельная AUC показывает качество ранжирования каждого класса; macro — среднее классовых AUC.

OvR-оценивание применимо и к совместно обученной softmax-модели.

Здесь OvR — способ оценивания. Переобучать отдельные бинарные модели не требуется.

Multiclass PR: тот же перенос бинарной задачи



Берём вероятность одного класса и меняем порог.

Строим его PR-кривую и вычисляем AP.

AP можно усреднить по классам; редкие классы полезно смотреть отдельно.

ROC и PR оценивают ранжирование. Классовые precision/recall/F1 — уже выбранные метки.

Метки и ранжирование отвечают на разные вопросы

Уже выбраны метки

Кого и с кем путаем?

Матрица ошибок, precision, recall, F1 и accuracy.

Есть классовые вероятности

Как объекты упорядочены по принадлежности классу?

ROC-AUC и PR/AP в бинарных постановках «класс против остальных».

Способ обучения модели и способ оценивания её выходов — разные решения.

А если правильных меток может быть несколько?

Multiclass: один класс

Основной отдел для обращения.

$$Y \in \{1, \dots, K\}$$

Multilabel: набор меток

Обращение: «оплата» и «безопасность».
Фильм: «драма» и «детектив».

$$Y \in \{0, 1\}^K$$

Один объект может иметь несколько верных меток одновременно.

Как работают multilabel-прогнозы

По отдельной бинарной модели на каждую метку.

$$q_c(x) = \sigma(s_c(x)), \qquad \hat{y}_c(x) = \mathbb{I}[q_c(x) \geq 0.5]$$

$$(0.85, 0.70, 0.15) \longrightarrow (1, 1, 0)$$

Вероятности меток не обязаны суммироваться в 1. Порог применяем к каждой отдельно.

Как оценивают multilabel

Precision, recall и F1 можно вычислять для каждой метки.

Exact match: совпал ли весь набор меток?

$$y = (1, 1, 0), \qquad \hat{y} = (1, 0, 0)$$

Две координаты совпали, но exact match не засчитывается.

Что изменилось в многометочной задаче

- Один объект допускает несколько правильных меток одновременно.
- Бинарные вероятности и пороги задают решение по каждой метке; сумма вероятностей не обязана равняться 1.
- Метрики по меткам показывают отдельные ошибки, exact match — совпадение всего набора.

Теперь посмотрим на входные признаки

Тип признака	Пример	Что известно о значениях
Числовой	Длина обращения	Количество, разности
Номинальный	Канал связи	Категории без порядка
Порядковый	Низкий / средний / высокий риск	Порядок, но не обязательно равные расстояния

Номера классов в y обозначали ответы. Категории в X нужно закодировать так, чтобы kNN или линейная модель могли использовать их как признаки.

Как передать канал обращения в модель?

Целочисленное кодирование категорий (label encoding): заменяем названия каналов условными номерами.

Chat → 0 | Email → 1 | Phone → 2

$$d(Chat, Email) = 1, \qquad d(Chat, Phone) = 2$$

Почему между каналами возникло такое отношение расстояний?

Числа были произвольными метками. Подставив их в признаки, мы навязали категориям количественную шкалу.

В scikit-learn `LabelEncoder` кодирует целевую y, `OrdinalEncoder` — категориальные признаки X.

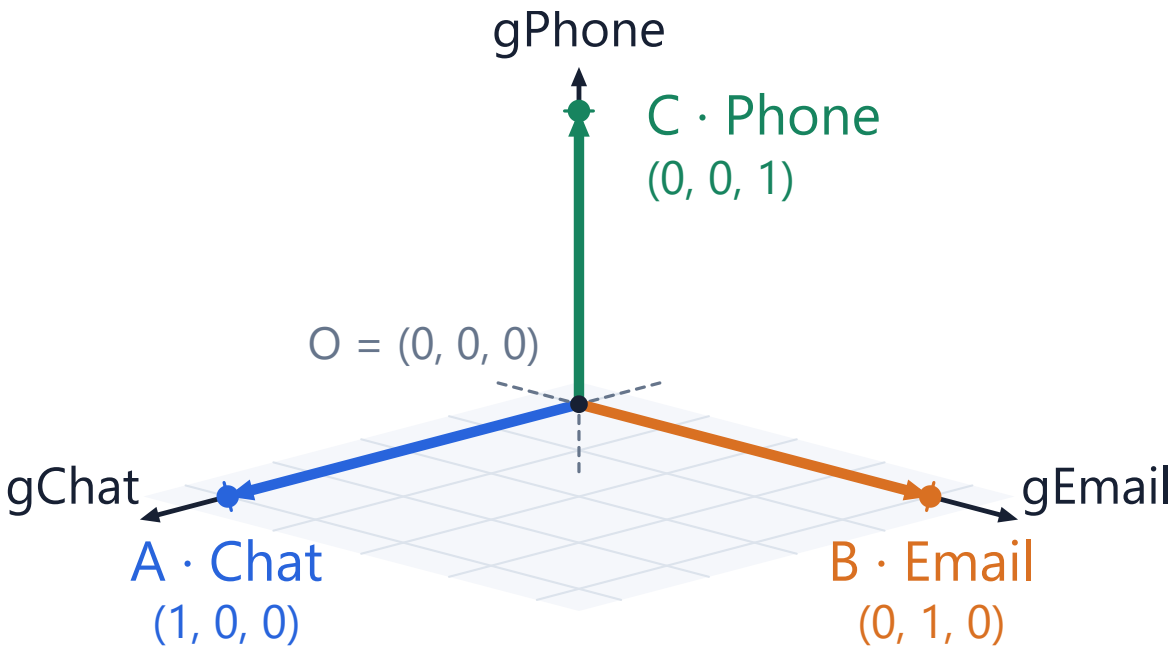
One-hot: отдельный индикатор для категории

Канал	Chat	Email	Phone
Chat	1	0	0
Email	0	1	0
Phone	0	0	1

$$g_j(x) = \mathbb{I}[f(x) = u_j]$$

Вместо условного номера создаём отдельный числовой индикатор: единица у своей категории, нули у остальных. Это dummy-переменные.

Геометрия полного one-hot



$$A = (1, 0, 0),$$
$$B = (0, 1, 0),$$
$$C = (0, 0, 1).$$

$$d(A, B) = d(A, C) = d(B, C) = \sqrt{2}$$

Все разные известные категории равноудалены.

Полный one-hot и евклидова метрика.
A — Chat, B — Email, C — Phone.

Зачем в линейной модели удаляют базовую dummy

При полном one-hot сумма индикаторов для каждого объекта равна 1:

$$g_{Chat}(x) + g_{Email}(x) + g_{Phone}(x) = 1$$

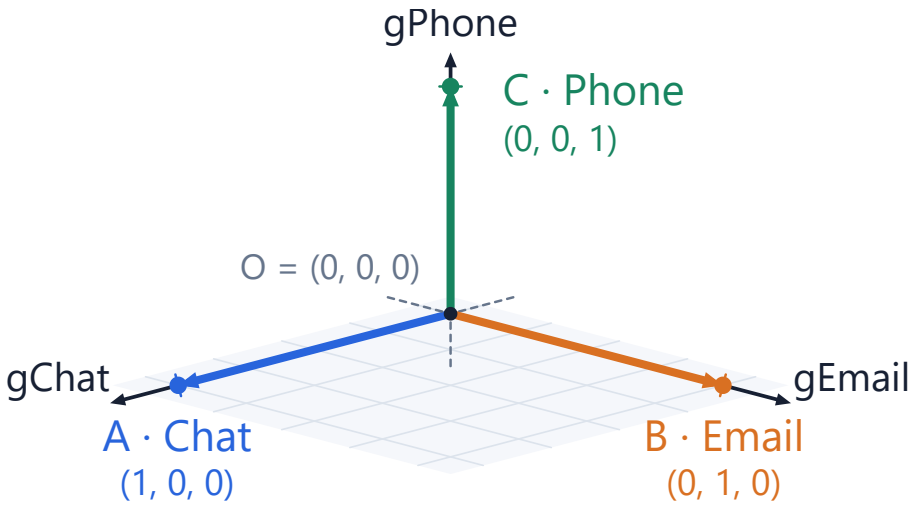
В модели со свободным членом уже есть столбец единиц. Полный набор dummy создаёт точную линейную зависимость столбцов.

Удаление одной базовой dummy устраняет эту зависимость. В обычной линейной модели без регуляризации множество возможных прогнозов сохраняется.

А что произойдёт с расстояниями для kNN?

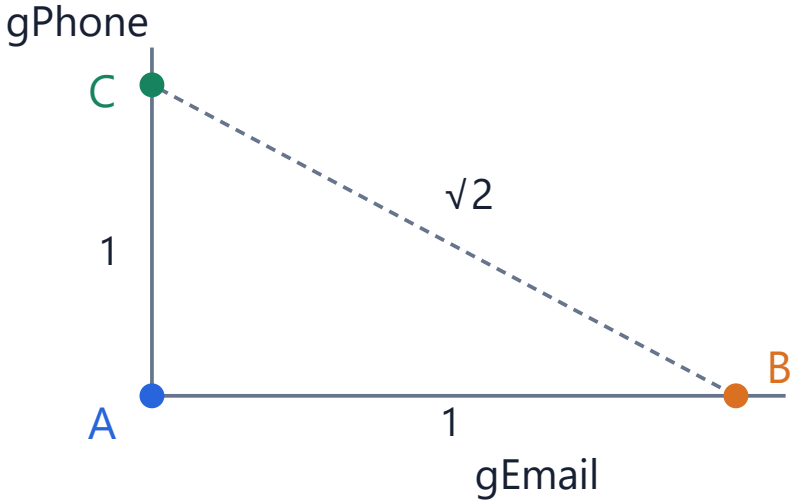
Полный one-hot → удалена базовая категория

Три индикатора



$$d(A,B) = d(A,C) = d(B,C) = \sqrt{2}$$

Удалили Chat, остаются Email и Phone



$$d(A,B) = d(A,C) = 1,$$
$$d(B,C) = \sqrt{2}$$

В kNN изменились расстояния. Могут измениться соседи и прогнозы.

Как объединить числовой признак и категорию?

Объект	Числовой z	Канал	One-hot
A	0	Chat	(1, 0, 0)
B	1	Email	(0, 1, 0)
C	2	Chat	(1, 0, 0)

z — числовой признак после выбранного масштабирования.

$$d^2_{\text{num}}(A,B) = (0-1)^2 = 1,$$
$$d^2_{\text{cat}}(A,B) = (1-0)^2 + (0-1)^2 + (0-0)^2 = 2,$$
$$d^2(A,B) = d^2_{\text{num}}(A,B) + d^2_{\text{cat}}(A,B) = 3.$$

$d^2(A,C) = 4 > 3 = d^2(A,B)$ — B ближе, хотя у C совпадает канал.

Пропуски и неизвестные категории

Числовые признаки

Заполнение пропусков
→ StandardScaler

Категориальные признаки

Заполнение пропусков
→ OneHotEncoder

↓ Объединяем в числовой вектор

Все обучаемые преобразования — только по соответствующей train-части.

Известны Chat, Email, Phone

Пришёл новый канал Messenger.

`handle_unknown="ignore"`

$$Messenger \longmapsto (0, 0, 0)$$

One-hot индикаторы обычно не масштабируют как числовые столбцы. Для kNN выбор представления и вклада групп признаков остаётся содержательным решением.

Кодировщик обработал новое значение без ошибки. Но это соглашение не сообщает модели реальное сходство Messenger с известными каналами.

Главные выводы

- kNN зависит от выбранного соседства и геометрии признаков.
- Для multiclass нужно различать построение прогнозов и оценивание ошибок.
- Кодирование категорий влияет на представление объектов и поведение модели.